

彩色图片分类

一 实验内容

本实验基于cifar10数据集使用tensorflow构建卷积神经网络，实现彩色图像分类

二 实验目标

- 了解CIFAR-10数据集
- 掌握卷积神经网络模型搭建、训练、预测、模型评估的完整流程 **重点**
- 熟练使用tensorflow.keras构建卷积神经网络 **重点**
- 熟练利用卷积神经网络解决图像分类任务 **难点**

三 实验环境

- 操作系统：ubuntu18 及以上
- 语言环境：Python3.6.5
- 编译器：jupyter notebook
- 深度学习环境：TensorFlow2.4.1
- 显卡（GPU）：NVIDIA GeForce RTX 3090

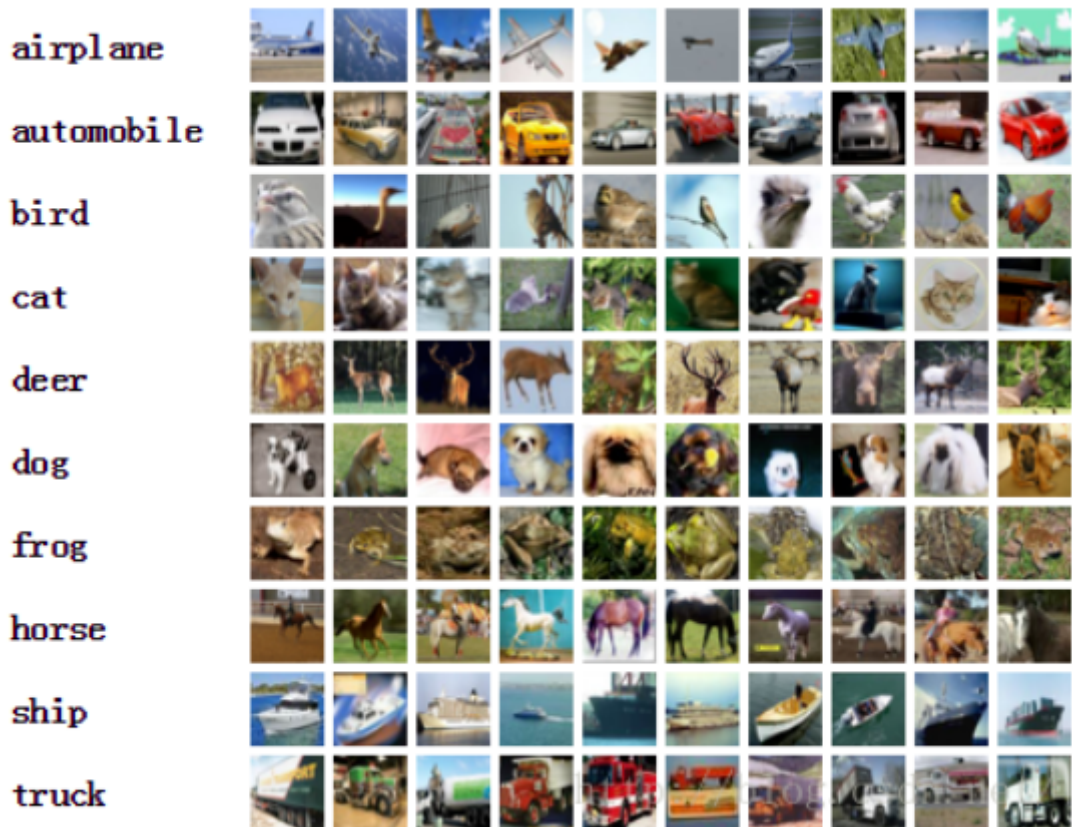
四 实验原理

1. CIFAR-10 数据集

CIFAR-10是一个常用的计算机视觉数据集，用于图像分类任务。它包含了10个不同类别的彩色图像，每个类别有6000张图像。数据集被分为训练集和测试集，其中训练集包含50000张图像，测试集包含10000张图像。

每个图像的尺寸为32x32像素，分为红色、绿色和蓝色三个通道。CIFAR-10的类别包括飞机、汽车、鸟类、猫、鹿、狗、青蛙、马、船和卡车。

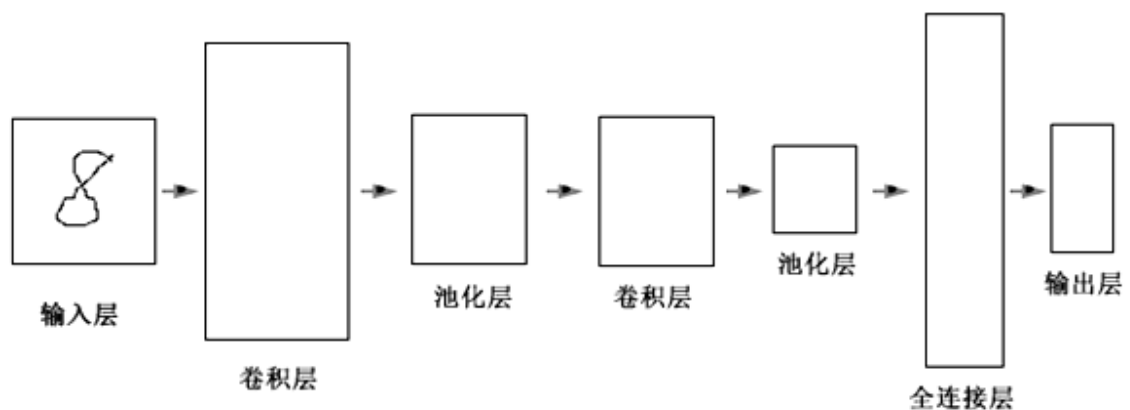
CIFAR-10数据集是一个挑战性的数据集，常用于测试和评估图像分类算法的性能。你可以使用该数据集来训练和测试你的机器学习模型，以实现图像分类任务。



2. 什么是CNN

CNN是用来处理多维数据的多层神经网络。CNN被认为是第一个真正成功的采用多层次结构网络的具有鲁棒性的深度学习方法。CNN通过挖掘数据中的空间上的相关性，来减少网络中的可训练参数的数量，达到改进前向传播网络的反向传播算法效率。CNN中层次之间的紧密联系和空间信息使得其特别适用于图像的处理和理解，并且能够自动的从图像中抽取丰富的相关特性。本实验通过Keras库接口实现CNN网络基础模型的搭建，并且显示网络模型的详细信息。后续我们将用搭建好的模型，借助预处理好的红绿灯图像数据，对其进行训练。最终使用训练好的模型，实现自动驾驶车辆识别红绿灯并且对其做出正确反应的效果。

卷积神经网络由输入层、卷积层、池化层、全连接层和输出层组成，其通过逐层处理的方式有效的提取输入数据的特征，尤其针对图像等二维数据输入。通过增加卷积层和池化层，还可以得到更加复杂的网络，其后的全连接层也可以采用多层结构，接下来我们介绍一下卷积神经网络的结构以及每一层的功能。



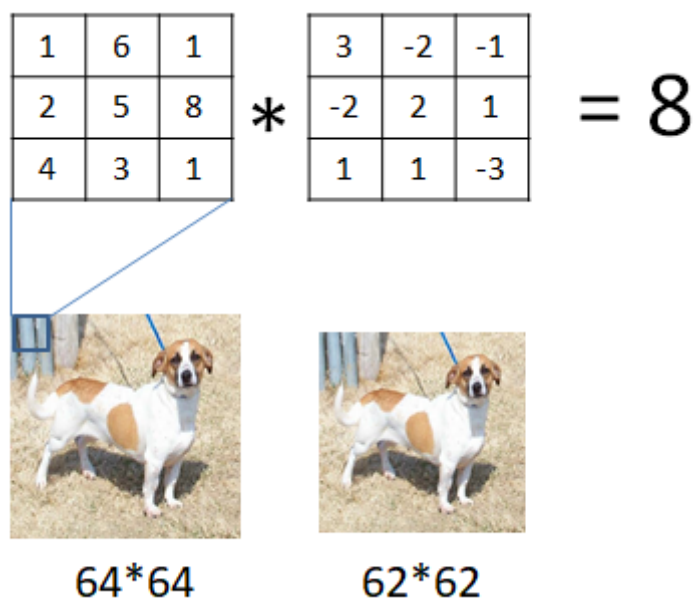
CNN基本结构

1. 输入层

这一层为数据的输入口，训练数据从这里传到整个卷积神经网络，为下一步卷积操作，提供数据。

2. 卷积层

这一层完成的是网络中的卷积操作，可以看做是输入样本和卷积核的内积运算，在进行完卷积操作后，得到的便是特征图。卷积层中是使用同一个卷积核对每一个输入样本进行卷积操作，第二层以及以后的卷积层，是把前一层的特征图作为输入数据，进行同样的卷积操作。如下图：



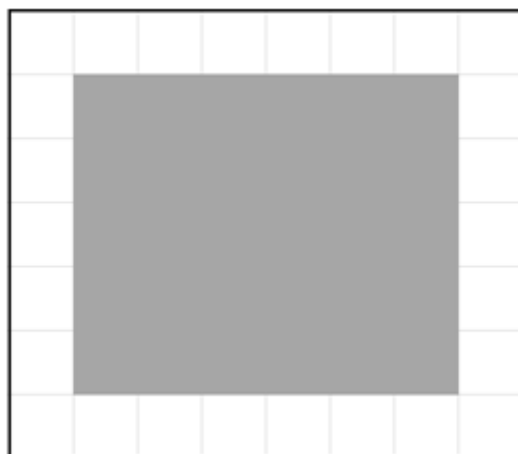
64*64



62*62

卷积操作示意图

对输入样本进行卷积计算后，可以将样本的某些特征更加清晰地表达出来，卷积在信号处理上的意义就是之前的信号衰减后对现在信号的影响，在图像处理的领域中，利用卷积可以将图片中的轮廓特征更明显的表达出来，这是因为图片中物体的轮廓处是色彩变化较大的位置，利用卷积我们可以将色彩变化不那么大的位置（例如背景纹理）模糊化处理，从而凸显出要识别的物体。同一个图片可以使用不同的卷积核处理，就可以得到不同方向上的轮廓特征，如下例中，将输入的图片二值化后得到的像素图用不同的卷积核进行内乘：



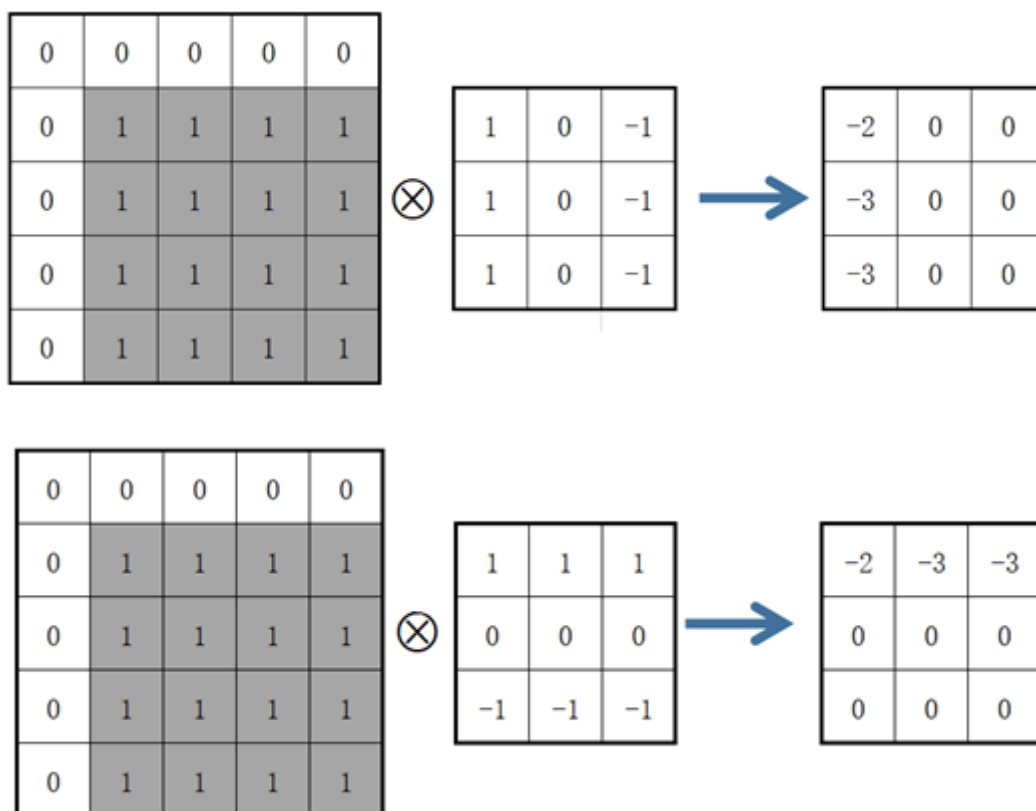
二值化图像

此处为简单示意，将上图左上角起5×5的部分转化为二值化栅格图如下：

0	0	0	0	0
0	1	1	1	1
0	1	1	1	1
0	1	1	1	1
0	1	1	1	1

5*5的二值化图像

分别与两个卷积核内乘结果如下：



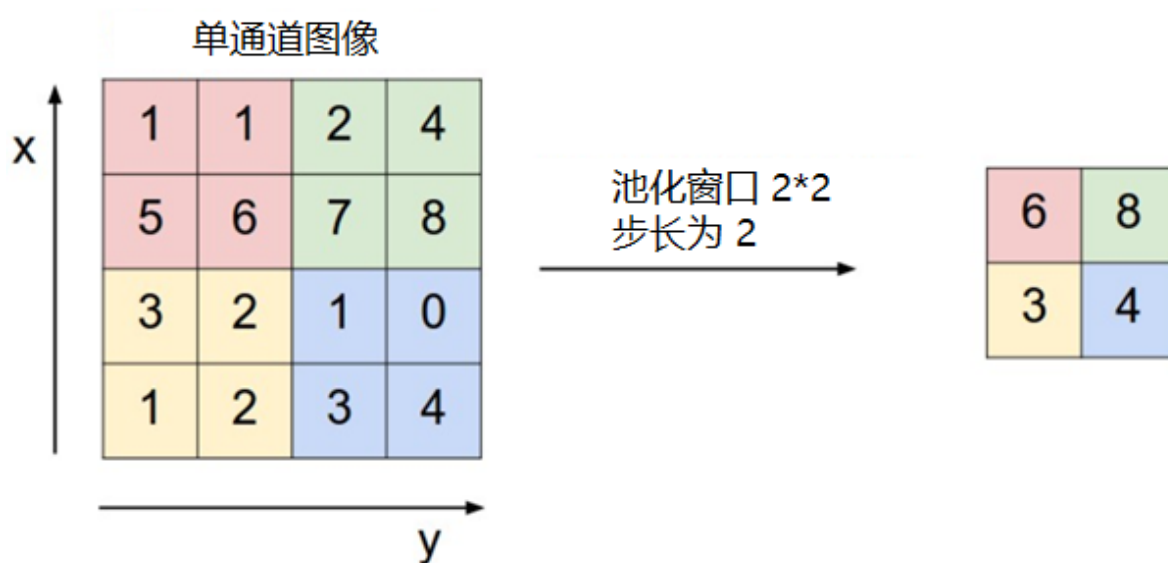
卷积操作

上面的两个卷积核的功能就是得到纵向轮廓和横向轮廓，可以从结果看出，水平方向的边界和垂直方向的边界都在边沿的3×3位置以内，这就是卷积对轮廓识别的功能，通过卷积运算，我们可以得到更清晰的边界数据来识别物体，特别是对三通道的彩色图片，分别对R (red) G (green) B (blue) 三色进行卷积运算，能得到对计算机而言辨识度更高的处理后图像。

我们可以发现，卷积内乘后图像尺寸会发生变化，对64×64的输入样本使用3×3的卷积核进行卷积运算后，得到62×62的特征图。特征图的尺寸会小于输入样本，为了得到与输入样本尺寸相同的特征图，可以使用0补充样本边界。上面的示意图卷积核的滑动步长为1，步长越大，卷积后的特征图越小。一个卷积层也可以有多个不同的卷积核，每个卷积核都对应一个特征图。此外，卷积结果不能直接作为特征图，需要通过激活函数运算后，把函数输出结果作为特征图。常见的激活函数包括sigmoid、tanh、ReLU等函数，这些函数都是非线性的函数，这里使用非线性函数是为了能表达能复杂的分类边界，如果不使用非线性函数，图像的卷积运算无非是矩阵的相乘变换，不具备识别复杂物体的能力，也就是说，线性函数的组合在高维空间并不能很好的对样本特征进行分割。

3. 池化层

池化层的作用是减小卷积层产生的特征图的尺寸。选择一个区域，根据该区域的特征图得到新特征图的这个过程称为池化。主要的池化方法有最大池化、平均池化、求和池化等，其中最常用的是最大池化，最大池化是选取图像区域内的最大值作为新的特征图，如下图所示：



池化层示意图

4. 全连接层

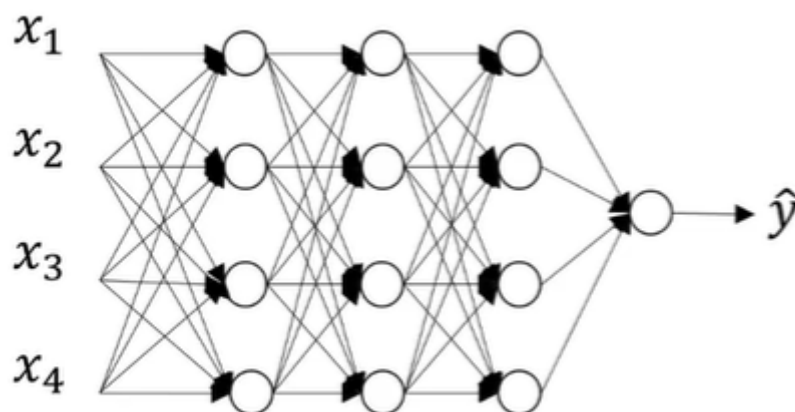
全连接层的功能是连接所有的特征，将输出值送给分类器（如softmax分类器），图像经过卷积层和池化层后得到的轮廓数据会在这一层得到判断，根据轮廓特征可以识别出各种结果的可能概率。

5. 输出层

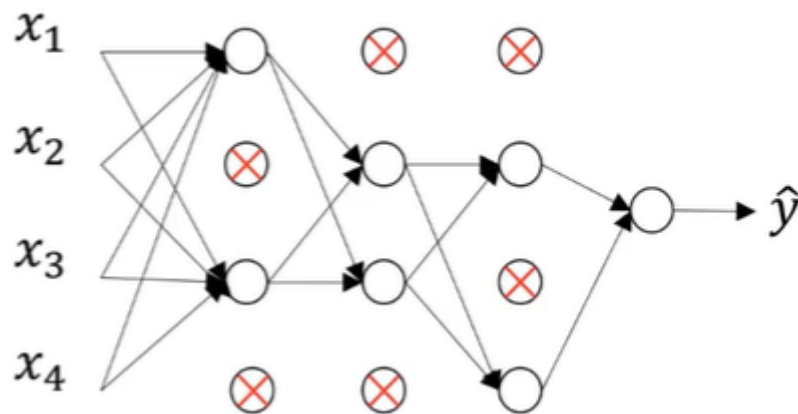
卷积神经网络的输出层是使用似然函数计算各类别的似然概率。使用softmax函数可以计算输出单元的似然概率，然后把概率最大的数字作为分类结果输出。

6. Dropout层

Dropout做的就是以一定的概率将每个隐层神经元的输出设置为零。以这种方式“dropped out”的神经元既不参与前向传播，也不参与反向传播。所以每次提出一个输入，该神经网络就尝试一个不同的结构，但是所有这些结构之间共享权重。因为神经元不能依赖于其他特定神经元而存在，所以这种技术降低了神经元复杂的互适应关系。正因如此，要被迫学习更为鲁棒的特征，这些特征在结合其他神经元的一些不同随机子集时有用。



原始神经网络

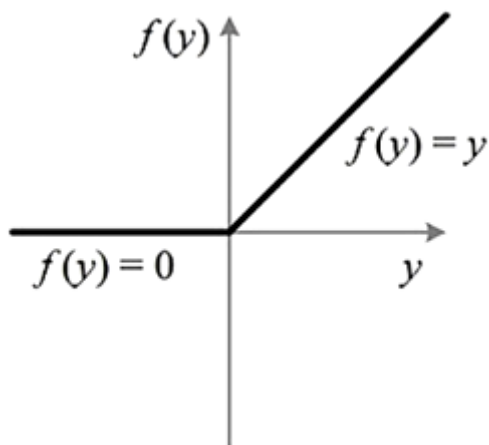


添加了Dropout层的神经网络

在训练时，随机选择部分神经元，使其置零，不参与本次优化迭代。每次减少参与优化迭代的神经元数目，使有效网络变小，防止过拟合，增强模型的泛化能力。

7. 非线性激活函数ReLU

在本实验中，我们使用非线性激活函数ReLU，ReLU函数图像如下：



ReLU图像

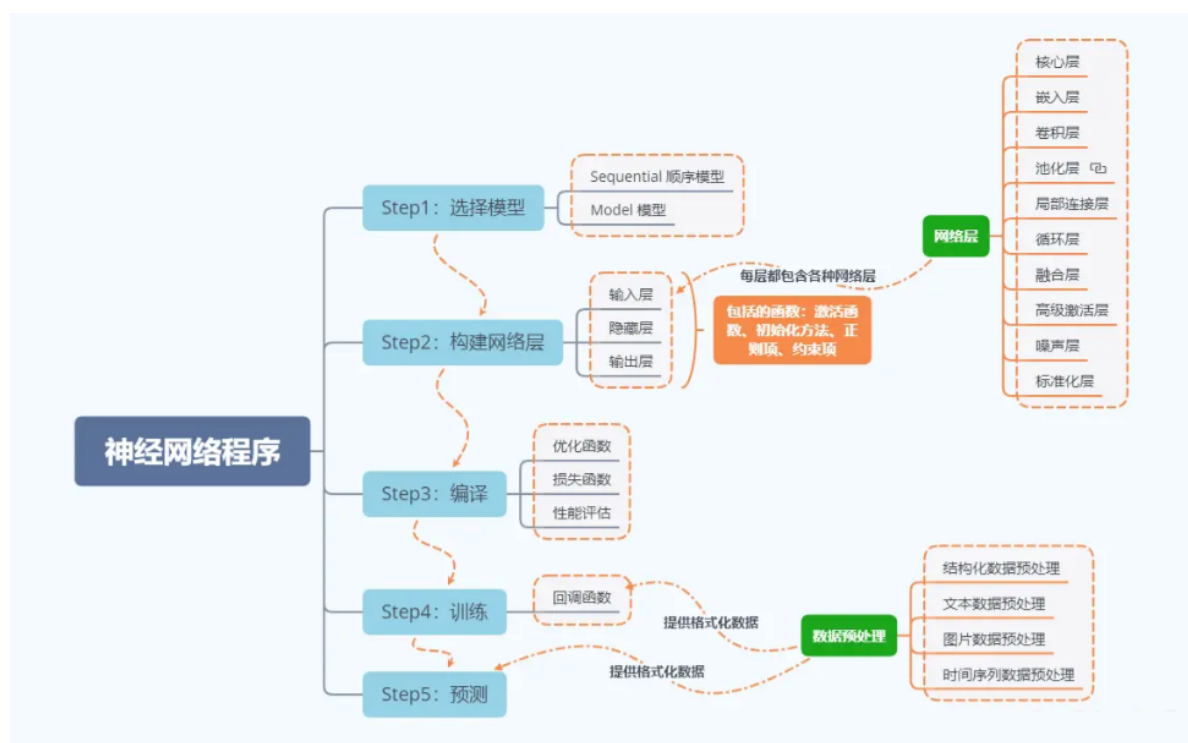
ReLU函数形式如下：

$$\text{ReLU} = \begin{cases} x & x > 0 \\ 0 & x \leq 0 \end{cases}$$

ReLU函数

从ReLU的图像可以看出，ReLU是分段线性函数，所有负值的函数值均为0，而正值得函数值不变，这种操作叫做单侧抑制，单侧抑制可以让网络中的神经元具有稀疏激活性，这样有利于模型更好的挖掘相关特征，拟合数据。使用ReLU激活函数，使得网络训练速度提高，同时解决了训练较深的网络时出现的一些问题，如梯度弥散问题等。

3. 神经网络程序说明



4. CNN优缺点

优点:

- 共享卷积核，优化计算量。
- 无需手动选取特征，训练好权重，即得特征。
- 深层次的网络抽取图像信息丰富，表达效果好。
- 保持了层级网络结构。
- 不同层次有不同形式与功能。

缺点:

- 需要调参，需要大样本量，GPU等硬件依赖。
- 物理含义不明确。

五 实验步骤

一、前期准备

设置GPU

如果使用的是CPU可以忽略这步

```

import tensorflow as tf
gpus = tf.config.list_physical_devices("GPU")

if gpus:
    gpu0 = gpus[0] #如果有多个GPU，仅使用第0个GPU
    tf.config.experimental.set_memory_growth(gpu0, True) #设置GPU显存用量按需使用
    tf.config.set_visible_devices([gpu0], "GPU")
  
```


导入数据

```
import tensorflow as tf
from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) =
datasets.cifar10.load_data()
```

归一化

```
# 将像素的值标准化至0到1的区间内。
train_images, test_images = train_images / 255.0, test_images / 255.0

train_images.shape, test_images.shape, train_labels.shape, test_labels.shape
```

打印结果:

```
((50000, 32, 32, 3), (10000, 32, 32, 3), (50000, 1), (10000, 1))
```

可视化

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog',
               'horse', 'ship', 'truck']

plt.figure(figsize=(20,10))
for i in range(20):
    plt.subplot(5,10,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i], cmap=plt.cm.binary)
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```

输出结果:



二、构建CNN网络

```
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)), #卷积
    layers.MaxPooling2D((2, 2)), #池化层1, 2*2采样
```



```

layers.Conv2D(64, (3, 3), activation='relu'), #卷积层2, 卷积核3*3
layers.MaxPooling2D((2, 2)), #池化层2, 2*2采样
layers.Conv2D(64, (3, 3), activation='relu'), #卷积层3, 卷积核3*3

layers.Flatten(), #Flatten层, 连接卷积层与全连接层
layers.Dense(64, activation='relu'), #全连接层, 特征进一步提取
layers.Dense(10) #输出层, 输出预期结果
])

model.summary() # 打印网络结构

```

输出结果:

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 30, 30, 32)	896
max_pooling2d (MaxPooling2D)	(None, 15, 15, 32)	0
conv2d_1 (Conv2D)	(None, 13, 13, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 6, 6, 64)	0
conv2d_2 (Conv2D)	(None, 4, 4, 64)	36928
flatten (Flatten)	(None, 1024)	0
dense (Dense)	(None, 64)	65600
dense_1 (Dense)	(None, 10)	650
Total params: 122,570		
Trainable params: 122,570		
Non-trainable params: 0		

二、编译

```

model.compile(optimizer='adam',

              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

```

三、模型训练

```
history = model.fit(train_images, train_labels, epochs=10,  
                    validation_data=(test_images, test_labels))
```

输出结果：

```
Epoch 1/10  
1563/1563 [=====] - 9s 4ms/step - loss: 1.7862 -  
accuracy: 0.3390 - val_loss: 1.2697 - val_accuracy: 0.5406  
Epoch 2/10  
1563/1563 [=====] - 5s 3ms/step - loss: 1.2270 -  
accuracy: 0.5595 - val_loss: 1.0731 - val_accuracy: 0.6167  
Epoch 3/10  
1563/1563 [=====] - 5s 3ms/step - loss: 1.0355 -  
accuracy: 0.6337 - val_loss: 0.9678 - val_accuracy: 0.6610  
Epoch 4/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.9221 -  
accuracy: 0.6727 - val_loss: 0.9589 - val_accuracy: 0.6648  
Epoch 5/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.8474 -  
accuracy: 0.7022 - val_loss: 0.8962 - val_accuracy: 0.6853  
Epoch 6/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.7814 -  
accuracy: 0.7292 - val_loss: 0.9124 - val_accuracy: 0.6873  
Epoch 7/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.7398 -  
accuracy: 0.7398 - val_loss: 0.8924 - val_accuracy: 0.6929  
Epoch 8/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.7008 -  
accuracy: 0.7542 - val_loss: 0.9809 - val_accuracy: 0.6854  
Epoch 9/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.6474 -  
accuracy: 0.7732 - val_loss: 0.8549 - val_accuracy: 0.7137  
Epoch 10/10  
1563/1563 [=====] - 5s 3ms/step - loss: 0.6041 -  
accuracy: 0.7889 - val_loss: 0.8909 - val_accuracy: 0.7046
```

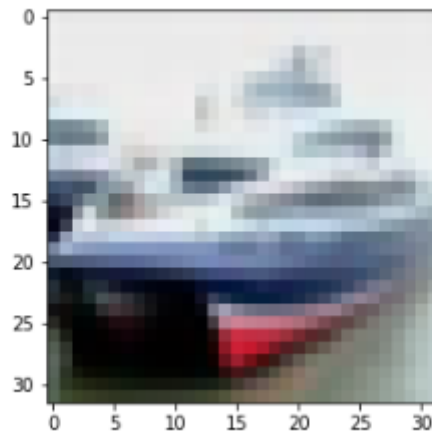
四、预测

通过模型进行预测得到的是每一个类别的概率，数字越大该图片为该类别的可能性越大

```
plt.imshow(test_images[1])
```

输出结果：

```
[10]: <matplotlib.image.AxesImage at 0x2278429a9a0>
```



```
import numpy as np

pre = model.predict(test_images)
print(class_names[np.argmax(pre[1])])
```

输出结果：

ship

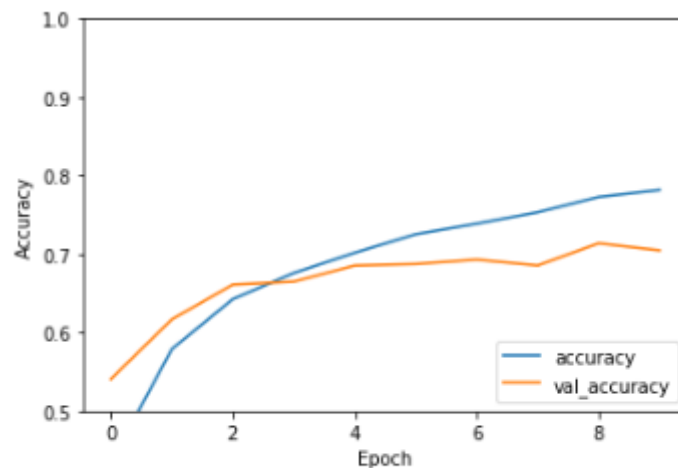
五、模型评估

```
import matplotlib.pyplot as plt

plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
plt.show()

test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
```

输出结果：



313/313 - 0s - loss: 0.8909 - accuracy: 0.7046

```
print(test_acc)
```

输出结果：

0.7045999765396118

六 实验总结

本实验需要学员掌握CNN的基本网络结构，数量使用tensorflow.keras构建各层网络结构，并将CNN灵活应用与解决分类问题，完整使用流程如下：

